

Розробка самовідновлювального конвеєру ETL та аналітичного сховища для аналізу текстових відгуків з використанням Apache Airflow і локальних великих мовних моделей

Виконав: **Мулик Андрій Васильович**

Керівник: **к.ф.-м.н., доц. Томка Юрій Ярославович**

Чернівці, 2026

РОЗДІЛ 1

Теоретичні основи

Якість даних
Self-Healing
Sentiment Analysis
Технологічний стек

РОЗДІЛ 2

Проектування системи

Архітектура
DuckDB
DAG Airflow
Модуль
самовідновлення
Blazor

РОЗДІЛ 3

Реалізація

Chain of Responsibility
TaskFlow API
Bearer Auth
Blazor WASM

РОЗДІЛ 4

Тестування та результати

Unit/System тести
Sentiment кореляція
Продуктивність
Порівняння

Актуальність

- Підприємства обробляють ~2,7 ПБ даних/місяць через конвеєрну інфраструктуру
- 41% часу інженерів даних — усунення проблем якості (DataOps 2023)
- Apache Airflow — щоденно у 55% організацій з оркестрацією
- LLaMA + Ollama: локальний LLM-інференс без хмарних провайдерів
- Відсутні україномовні дослідження self-healing конвеєрів з LLM

Мета та завдання

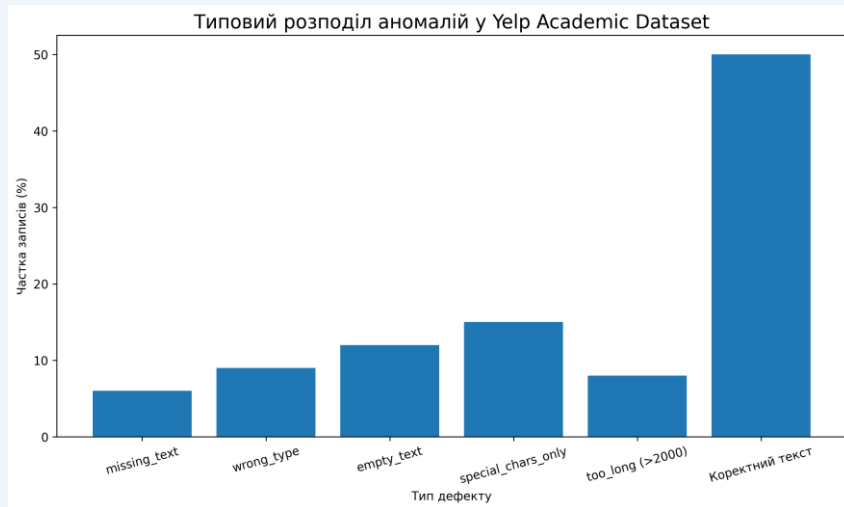
Мета:

Розробити самовідновлювальний ETL-конвеєр аналізу тональності з вбудованою логікою автоматичного виправлення дефектів, локальним LLM (Llama3.2/Ollama) та Blazor-дашбордом моніторингу.

Завдання:

1. Теоретичне дослідження self-healing та sentiment analysis
2. Аналіз якості даних Yelp Academic Dataset
3. Проектування багаторівневої архітектури системи
4. Реалізація детерміністичного алгоритму самовідновлення
5. Інтеграція Llama3.2 через Ollama (локальний інференс)

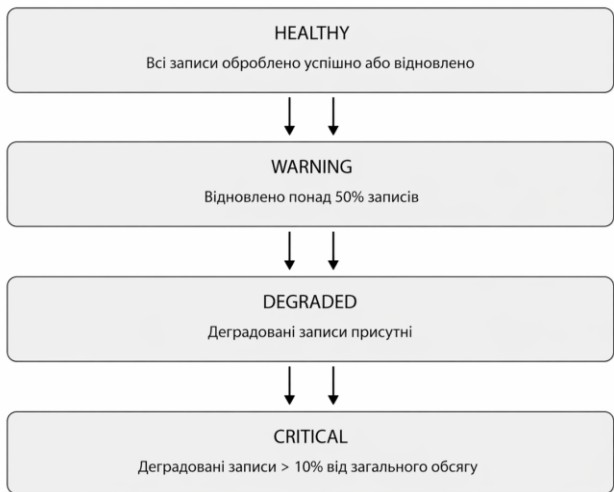
Тип дефекту	Стратегія виправлення	Код статусу
missing_text	Placeholder "No review text provided."	healed
wrong_type	Примусове перетворення str()	healed
empty_text	Заповнення placeholder-текстом	healed
special_chars_only	Заміна на "[Non-text content]"	healed
too_long (>2000)	Обрізка до 2000 символів + "..."	healed
Коректний текст	Нормалізація пробілів strip()	success



[Рис. 1.1 – Типовий розподіл аномалій у датасеті]

41%

часу інженерів даних витрачається
на усунення проблем якості
(DataOps Impact Assessment, 2023)



[Рис. 1.2 – Схема Self-Healing конвеєра]

HEALTHY — Всі записи успішно оброблено**WARNING** — Відновлено >50% записів**DEGRADED** — Є деградовані записи**CRITICAL** — Деградовані >10% від обсягу

Підхід	Переваги	Недоліки
Rule-based	Передбачуваність, швидкість	Ручне оновлення правил
ML-based (RL)	Адаптивність до нових сценаріїв	Потребує великих даних
Гібридний *	Баланс між підходами	Складніша архітектура

* Обраний підхід у даній системі: детерміністична логіка + чотири рівні здоров'я

- Рівень безперервного моніторингу (real-time)
- Механізм діагностики та класифікації помилок
- Реєстр стратегій виправлення (5 класів дефектів)
- Компонент звітності health-звіту (JSON)

Еволюція методів Sentiment Analysis

2000-ті	Lexicon-based (VADER, SentiWordNet)	Не враховує контекст
2010-ті	ML: Naive Bayes, SVM, TF-IDF	Не враховує залежності
2015+	Deep: LSTM, CNN + Attention	Великий датасет
2018+	Transformers: BERT, RoBERTa	Великий розмір моделі
2024+	LLM: llama3.2 + Ollama (цей проєкт)	

Apache Airflow 3.0

Оркестрація DAG, TaskFlow API, Param API, Web UI

Ollama + llama3.2

Локальний LLM-інференс, REST API, 3B параметрів

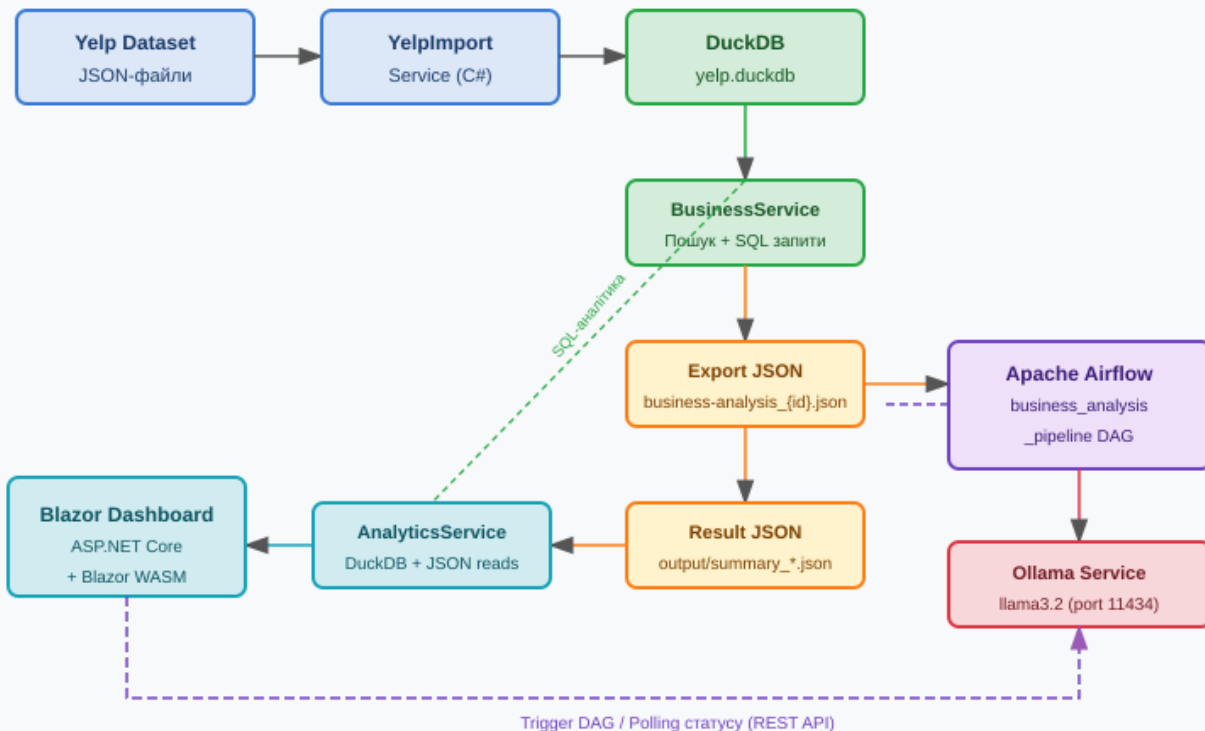
DuckDB

Колонкове аналітичне сховище, вбудоване, субсекундні запити

Blazor Hosted WASM

Єдина мова C# для сервера і клієнта, MudBlazor UI

Загальна архітектура системи Self-Healing Pipeline

**Рівень вхідних даних**

Yelp Academic Dataset (NDJSON) → Blazor Export Service

Оркестрація

Apache Airflow 3.0 · DAG
business_analysis_pipeline

Обробка даних

Self-Healing Module (Chain of Responsibility) +
LLM Sentiment

Зберігання

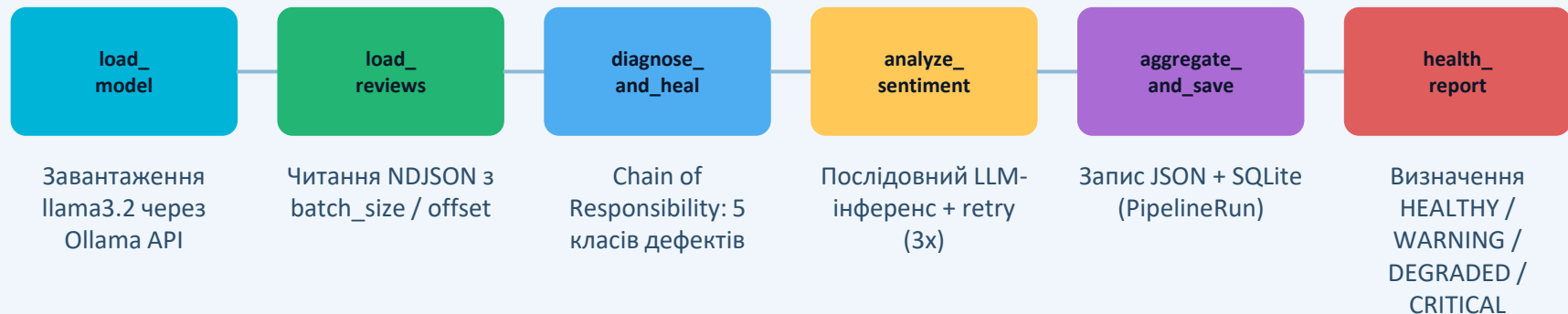
DuckDB (аналітика, 6+ ГБ) · SQLite (Auth,
Pipeline runs)

Презентація

Blazor Hosted WebAssembly · MudBlazor · Real-time polling

[Рис. 2.1 – Багаторівнева архітектура системи]

Проектування конвеєрів Apache Airflow



[Рис. 2.3 – Структура DAG `business_analysis_pipeline`]

Параметри запуску (Param API)

input_file

Шлях до NDJSON (рядок)

batch_size

Розмір пакету (0 = всі)

ollama_model

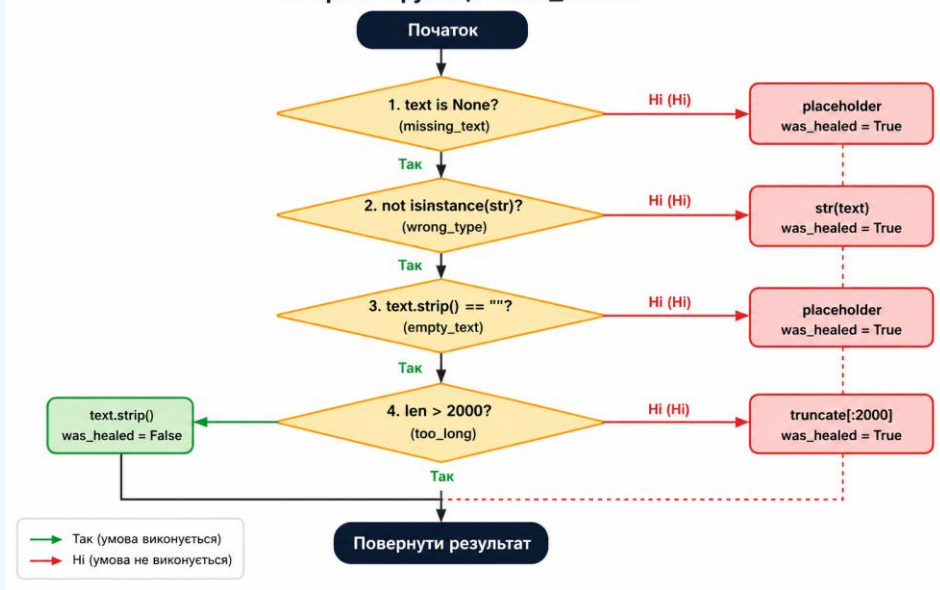
Ім'я моделі (llama3.2)

offset

Зміщення для відновлення

Ланцюжок відповідальності (_heal_review)

Алгоритм функції heal_review



text is None? → filled_with_placeholder

not isinstance(text,str)? → type_conversion → str(text)

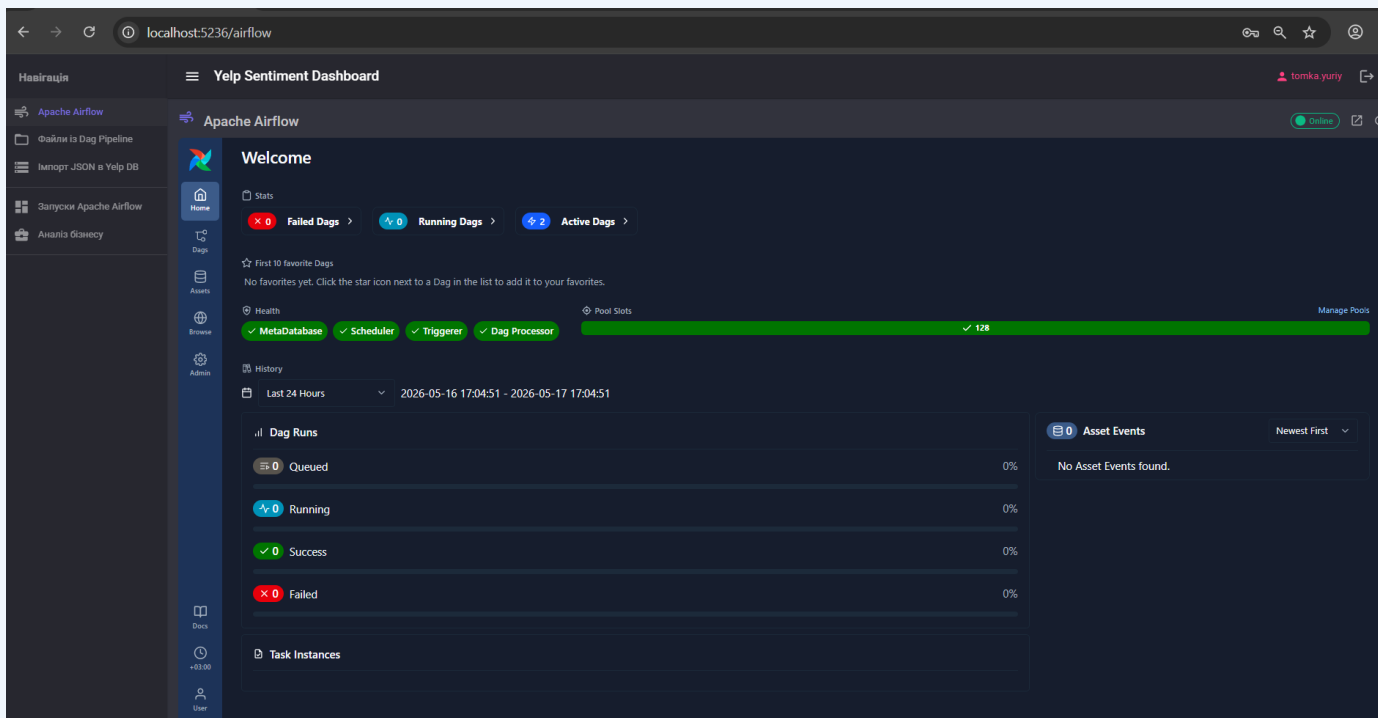
not text.strip()? → filled_with_placeholder

немає a-zA-Z0-9? → replaced → "[Non-text content]"

len(text)>2000? → truncated_text + "..."

Валідний текст → text.strip() → status: success

original_text зберігається у кожному записі до будь-якого виправлення



[Рис. 2.7 – Архітектура Blazor Hosted WebAssembly дашборду]

Сторінки дашборду

Home.razor

Список запусків + health-чіпи

RunDetail.razor

Pie chart, bar chart, Healing Report

BusinessAnalysis.razor

Heatmap, тренди, real-time polling

Login.razor

Bearer-автентифікація

Архітектурні рішення: Blazor Hosted WebAssembly · MudBlazor · PeriodicTimer 5с · Task.WhenAll (8 ендпоінтів паралельно)

Порівняння Apache Airflow з аналогами

Критерій	Apache Airflow 3.0	Prefect 3.x	Dagster
Парадигма задач	Python DAG + TaskFlow API	Flow + @task	Asset-first + ops
Обробка помилок	retry, on_failure_callback	Circuit breakers, SLA	Asset checks, hooks
Self-healing	Ручна реалізація ✓	Ручна реалізація	Вбудовані asset checks
Моніторинг UI	Вбудований Web UI	Prefect Cloud/Server	Dagster UI (Dagit)
GitHub stars	36 000+	15 000+	11 000+
Self-hosted	Повністю ✓	Self-hosted / Cloud	Self-hosted / Cloud
Придатність	★★★★★	★★★★☆	★★★★☆

Self-Healing Module (Python)

- Chain of Responsibility: 5 умовних гілок у `_heal_review()`
- Null Object Pattern: `_degraded_results()` при недоступності Ollama
- Зберігання `original_text` у кожному записі

Apache Airflow Pipeline

- TaskFlow API: `@task`-декоратори, залежності через повернені значення
- Param API: типізовані параметри `input_file`, `batch_size`, `offset`
- `batch_runner.py`: послідовний/паралельний запуск (`ThreadPoolExecutor`)

Рівень зберігання (DuckDB + SQLite)

- `read_json_auto()`: потоковий імпорт NDJSON без C#-десеріалізації
- `TRY_CAST` для безпечного читання нестандартних дат
- `JOIN` з "user" (у лапках — зарезервоване слово DuckDB)

Автентифікація та Blazor

- PBKDF2-SHA256 (100 000 ітерацій) для зберігання паролів
- `TokenAuthHandler`: власна Bearer-схема без зовнішніх залежностей
- `PeriodicTimer` + `Task.WhenAll`: 4-кратне покращення часу відгуку

Аналіз бізнесу

Джерело: `ye1p.duckdb`

 ВИБІР БІЗНЕСУ

 ЗАПУСК DAG

 АНАЛІТИКА

Топ 100 за кількістю відгуків

Royal House

New Orleans, LA · ★ 4,0 · 5070 відгуків

Commander's Palace

New Orleans, LA · ★ 4,5 · 4876 відгуків

Luke

New Orleans, LA · ★ 4,0 · 4554 відгуків

Cochon

New Orleans, LA · ★ 4,0 · 4421 відгуків

Pat's King of Steaks

Philadelphia, PA · ★ 3,0 · 4250 відгуків

Biscuit Love: Gulch

Nashville, TN · ★ 4,0 · 4207 відгуків

Pappy's Smokehouse

Saint Louis, MO · ★ 4,5 · 3999 відгуків

Felix's Restaurant & Oyster Bar

New Orleans, LA · ★ 4,0 · 3966 відгуків

Gumbo Shop

 Biscuit Love: Gulch

 Nashville, TN

★ 4,0 / 5.0

★★★★★

 4 207 відгуків

 Burgers, American (Traditional), Breakfast & Brunch, Restaurants, Southern

  Файл створено успішно!

 Назва файлу: `business-analysis_1b5mnK8bMnnju_cvU65GqQ.json`

 Шлях: `C:\Git\SelfHealingPipeline\airflow-pipeline\input\business-analysis_1b5mnK8bMnnju_cvU65GqQ.json`

 Записів: 4 247 відгуків

 Створено: 2026-04-22 20:42:49

 КОПІЮВАТИ ШЛЯХ

 ПОВТОРИТИ

  ПЕРЕЙТИ ДО ЗАПУСКУ DAG →

Аналіз бізнесу

Джерело: yelp.duckdb

ВИБІР БІЗНЕСУ ▶ ЗАПУСК DAG || АНАЛІТИКА

Файл: business-analysis_1b5mnK8bMnnju_cvU65GqQ.json

Відгуків: 4 247

Статус: success Run ID: manual__2026-04-22T17:43:26.027998+00:00

Прогрес аналізу

100%

Оброблено 29 з 29 відгуків

Кроки DAG

- ✓ load_model (success)
- ✓ load_reviews (success)
- ✓ diagnose_and_heal (success)
- ✓ analyze_sentiment (success)
- ✓ aggregate_and_save (success)
- ✓ health_report (success)

Лог analyze_sentiment

```
{
  "content": [
    {
      "event": "group::Log message source details",
      "sources": [
        "/opt/airflow/logs/dag_id=business_analysis_pipeline/run_id>manual__2026-04-22T17:43:26.027998+00:00/task_id=analyze_sentiment/attempt=1.log"
      ]
    },
    {
      "event": "endgroup::",
      "timestamp": "2026-04-22T17:45:07.412265Z",
      "event": "DAG bundles loaded: dags-folder",
      "level": "info",
      "logger": "airflow.dag_processing.bundles.manager.DagBundlesManager",
      "filename": "manager.py",
      "lineno": 209,
      "timestamp": "2026-04-22T17:45:07.418720Z",
      "event": "Filling up the DagBag from /opt/airflow/dags/business_pipeline_dag.py",
      "level": "info",
      "logger": "airflow.dag_processing.dagbag.BundleDagBag",
      "filename": "dagbag.py",
      "lineno": 469,
      "timestamp": "2026-04-22T17:45:07.693212Z",
      "event": "Analyzing 29 reviews...",
      "level": "info",
      "logger": "unusual_prefix_f8ffe688237f29048f7ecd647f726351aff1b042_business_pipeline_dag",
      "filename": "business_pipeline_dag.py",
      "lineno": 316,
      "timestamp": "2026-04-22T17:46:47.427377Z",
      "event": "Analyzed 20/29 reviews.",
      "level": "info",
      "logger": "unusual_prefix_f8ffe688237f29048f7ecd647f726351aff1b042_business_pipeline_dag",
      "filename": "business_pipeline_dag.py",
      "lineno": 223,
      "timestamp": "2026-04-22T17:48:25.305200Z",
      "event": "Analyzed 20/29 reviews.",
      "level": "info",
      "logger": "unusual_prefix_f8ffe688237f29048f7ecd647f726351aff1b042_business_pipeline_dag",
      "filename": "business_pipeline_dag.py",
      "lineno": 223,
      "timestamp": "2026-04-22T17:49:52.816584Z",
      "event": "Analyzed 29/29 reviews.",
      "level": "info",
      "logger": "unusual_prefix_f8ffe688237f29048f7ecd647f726351aff1b042_business_pipeline_dag",
      "filename": "business_pipeline_dag.py",
      "lineno": 223,
      "timestamp": "2026-04-22T17:49:52.817754Z",
      "event": "Done. Returned value was: [{'review_id': 'HrhKYdHYhnrrBm6HH5rnkg', 'business_id': '1b5mnK8bMnnju_cvU65GqQ', 'stars': 5, 'text': 'Great friendly service. Delicious food. Cute atmosphere. Bonuts are wonderful. I also had the sausage gravy biscuit which was great.', 'original_text': 'Great friendly service. Delicious food. Cute atmosphere. Bonuts are wonderful. I also had the sausage gravy biscuit which was great.', 'predicted_sentiment': 'POSITIVE', 'confidence': 1.0}]"
    }
  ]
}
```

[Рис. 4.2 – Успішне виконання DAG в Airflow UI]

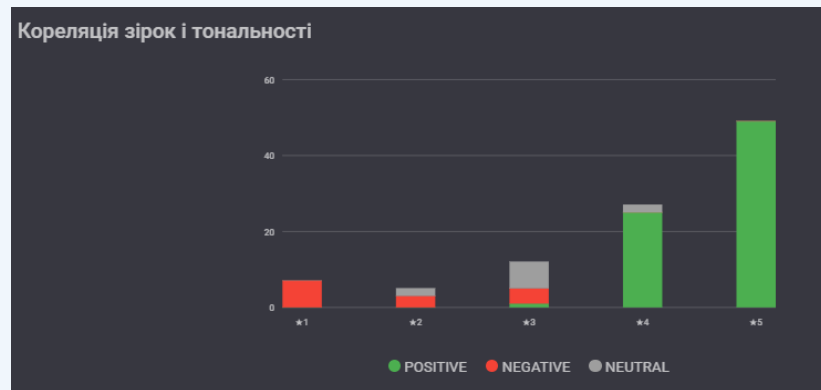
Сценарій	degraded_rate	healing_rate	Очікуваний статус	Фактичний статус	Результат
Всі записи успішно оброблені	0.0	0.0%	HEALTHY	HEALTHY	Відповідає
15% записів потребували healing	0.0	15%	HEALTHY	HEALTHY	Відповідає
55% записів потребували healing	0.0	55%	WARNING	WARNING	Відповідає
5% деградованих записів	5%	будь-яке	DEGRADED	DEGRADED	✓ Відповідає
15% деградованих записів	15%	будь-яке	CRITICAL	CRITICAL	✓ Відповідає

Таблиця 4.3 – Тестування health-статусів конвеєру (TC-08, TC-09)

Тестування підтвердило коректність логіки визначення health-статусу для всіх граничних значень. Порогові умови (degraded > 10% → CRITICAL, healing > 50% → WARNING) реалізовані точно відповідно до специфікації розділу 2.6.

Зірки	POSITIVE (%)	NEGATIVE (%)	NEUTRAL (%)	Кореляція
1 ★	8–12%	75–82%	10–15%	Висока
2 ★	15–22%	55–65%	18–25%	Середня
3 ★	25–35%	20–30%	40–50%	Низька
4 ★	60–70%	8–15%	18–28%	Висока
5 ★	78–88%	3–7%	8–15%	Висока

3-зіркові відгуки мають найнижчу кореляцію через змішаний емоційний характер (expected).



[Рис. 4.3 – Розподіл тональності та кореляція зірок]

Confidence score за статусами:

Статус	Avg confidence
success	0.78–0.85
healed	0.65–0.75
degraded	0.50 (fixed)

Час виконання DAG залежно від batch_size (llama3.2, CPU Ryzen 5800H):

batch	load_model	diagnose	sentiment	Загалом
10	~8 с	<1 с	~50 с	~60 с
100	~8 с	<1 с	~500 с	~510 с
500	~8 с	~2 с	~2500 с	~2510 с
1000	~8 с	~4 с	~5000 с	~5016 с

analyze_sentiment: 96–98% часу виконання. ~5 с/відгук на CPU, ~0.5–1 с на GPU.

DuckDB vs прямий JSON-scan:

Запит	DuckDB	JSON	×
Пошук бізнесу (ILIKE)	8–15 мс	4000–8000 мс	~400
Місячний тренд	12–25 мс	5000–12000 мс	~400
Heatmap 7×24	18–35 мс	8000–15000 мс	~300

Self-Healing**Переваги**

Детерміністична логіка, повний audit trail кожного виправлення, 5 класів помилок

Обмеження

Відсутня підтримка нових типів без змін коду DAG

LLM Inference**Переваги**

Конфіденційність даних (локально); нульова вартість API

Обмеження

Послідовний інференс ~5 с/відгук на CPU; відсутня GPU-підтримка у Docker

DuckDB**Переваги**

Субсекундні аналітичні запити на 6+ ГБ без зовнішнього сервера

Обмеження

Відсутня реплікація та паралельний запис між процесами

Blazor WASM**Переваги**

Єдина мова C# для сервера і клієнта; MudBlazor-компоненти

Обмеження

Перше завантаження WASM ~3–5 с; XSS-ризик localStorage

ВИСНОВКИ

- 1 Розроблено самовідновлювальний ETL-конвеєр із детерміністичним алгоритмом виявлення та виправлення 5 класів дефектів вхідних даних на базі Apache Airflow 3.0
- 2 Реалізовано інтеграцію локальної LLM llama3.2 через Ollama із retry-логікою та Null Object Pattern (graceful degradation при недоступності Ollama)
- 3 Побудовано аналітичне сховище DuckDB з колонковим зберіганням — субсекундні запити (300–500× швидше прямого JSON-сканування) на датасеті 6+ ГБ
- 4 Тестування підтвердило: середній confidence ≥ 0.78 на success-записах; очікувана кореляція зірок/тональності (найвища для 1★ та 5★)
- 5 Apache Airflow 3.0 обґрунтовано обраний як найзріліший оркестратор (36 000+ GitHub stars) з повною self-hosted моделлю та Param API